Task 1

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        printf("Child Process: PID = %d, Parent PID = %d\n", getpid(), getppid());
    } else {
        printf("Parent Process: PID = %d, Created Child PID = %d\n", getpid(), pid);
    }
    return 0;
}
```

Task 2

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
```

```
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child executing task...\n");
        sleep(2);
        printf("Child exiting with code 42\n");
        _exit(42);
    } else {
        int status;
        printf("Parent waiting for child...\n");
        wait(&status);
        if (WIFEXITED(status)) {
            printf("Parent: Child terminated with exit status %d\n", WEXITSTATUS(status));
        }
    }
    return 0;
}
```

Task 3

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
```

```
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child PID: %d terminating now\n", getpid());
        _exit(0);
    } else {
        printf("Parent sleeping for 10s without wait(). Inspect via 'ps -l'\n");
        sleep(10);
        wait(NULL);
        printf("Parent cleaned up child. Exiting.\n");
    }
    return 0;
}
```

Task 4

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Initial Parent PID: %d\n", getppid());
        sleep(3);
```

```
        printf("After parent dies, new Parent PID: %d\n", getppid());
        _exit(0);
    } else {
        printf("Parent PID: %d exiting immediately\n", getpid());
        exit(0);
    }
}
```

Task 5

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int shared_counter = 100;

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        shared_counter += 50;
        printf("Child sees shared_counter = %d\n", shared_counter);
        _exit(0);
    } else {
```

```
        wait(NULL);
        printf("Parent sees shared_counter = %d\n", shared_counter);
    }
    return 0;
}
```

Task 6

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child replacing its binary with '/bin/ls'...\n");
        char *args[] = {"ls", "-l", NULL};
        execv("/bin/ls", args);
        perror("execv failed");
        _exit(127);
    } else {
        wait(NULL);
        printf("Parent reaped replaced child image\n");
```

```
    }
    return 0;
}
```

Task 7

```
#include <stdio.h>
#include <pthread.h>

#define N 3

int matrix[N][N] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
int vector[N] = {1, 2, 3};
int result[N];

void *worker_multiply_row(void *arg) {
    int row = *(int *)arg;
    int sum = 0;
    for (int col = 0; col < N; col++) {
        sum += matrix[row][col] * vector[col];
    }
```

```
    result[row] = sum;
    return NULL;
}

int main() {
    pthread_t threads[N];
    int rows[N] = {0, 1, 2};

    for (int i = 0; i < N; i++) {
        pthread_create(&threads[i], NULL, worker_multiply_row, &rows[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Result Vector: [%d, %d, %d]\n", result[0], result[1], result[2]);
    return 0;
}
```

Task 8

```
#include <stdio.h>
#include <pthread.h>
```

```
volatile long counter = 0;

void *increment_task(void *arg) {
   for (int i = 0; i < 100000; i++) {
      counter++;
   }
   return NULL;
}

int main() {
   pthread_t t1, t2;

   pthread_create(&t1, NULL, increment_task, NULL);
   pthread_create(&t2, NULL, increment_task, NULL);

   pthread_join(t1, NULL);
   pthread_join(t2, NULL);

   printf("Expected: 200000 | Actual counter: %ld\n", counter);
   return 0;
}
```

Task 9

```
#include <stdio.h>
```

```
#include <pthread.h>

long counter = 0;
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;

void *safe_increment(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&mtx);
        counter++;
        pthread_mutex_unlock(&mtx);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, safe_increment, NULL);
    pthread_create(&t2, NULL, safe_increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter with Mutex: %ld\n", counter);
    return 0;
```

}

Task 10

Task 10: Producer-Consumer using Condition Variables

```c
#include <stdio.h>
#include <pthread.h>

#define CAPACITY 3

int buffer[CAPACITY];
int count = 0;
int in = 0;
int out = 0;

pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    for (int item = 1; item <= 5; item++) {
        pthread_mutex_lock(&mtx);
        while (count == CAPACITY) {
```

```
            pthread_cond_wait(&not_full, &mtx);
        }
        buffer[in] = item;
        in = (in + 1) % CAPACITY;
        count++;
        printf("Produced: %d\n", item);
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mtx);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= 5; i++) {
        pthread_mutex_lock(&mtx);
        while (count == 0) {
            pthread_cond_wait(&not_empty, &mtx);
        }
        int item = buffer[out];
        out = (out + 1) % CAPACITY;
        count--;
        printf("Consumed: %d\n", item);
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mtx);
    }
```

```
    return NULL;
}

int main() {
    pthread_t p, c;

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

Task 11: Reader-Writer Problem

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int shared_data = 0;
pthread_rwlock_t rw_lock = PTHREAD_RWLOCK_INITIALIZER;
```

```
void *writer(void *arg) {
    pthread_rwlock_wrlock(&rw_lock);
    shared_data = 42;
    printf("Writer 1 updated value to: %d\n", shared_data);
    pthread_rwlock_unlock(&rw_lock);
    return NULL;
}

void *reader(void *arg) {
    int id = *(int *)arg;
    sleep(1);
    pthread_rwlock_rdlock(&rw_lock);
    printf("Reader %d read value: %d\n", id, shared_data);
    pthread_rwlock_unlock(&rw_lock);
    return NULL;
}

int main() {
    pthread_t w, r1, r2;
    int id1 = 1, id2 = 2;

    pthread_create(&w, NULL, writer, NULL);
    sleep(1);
    pthread_create(&r1, NULL, reader, &id1);
    pthread_create(&r2, NULL, reader, &id2);
```

```
    pthread_join(w, NULL);
    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
    return 0;
}
```

Task 12: Simulating Deadlock Conditions

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutexA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutexB = PTHREAD_MUTEX_INITIALIZER;

void *thread1_work(void *arg) {
    pthread_mutex_lock(&mutexA);
    printf("Thread 1 acquired Lock A, waiting for Lock B...\n");
    sleep(1);
    pthread_mutex_lock(&mutexB);
    pthread_mutex_unlock(&mutexB);
    pthread_mutex_unlock(&mutexA);
```

```
    return NULL;
}

void *thread2_work(void *arg) {
    pthread_mutex_lock(&mutexB);
    printf("Thread 2 acquired Lock B, waiting for Lock A...\n");
    sleep(1);
    pthread_mutex_lock(&mutexA);
    pthread_mutex_unlock(&mutexA);
    pthread_mutex_unlock(&mutexB);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1_work, NULL);
    pthread_create(&t2, NULL, thread2_work, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

This program will block indefinitely due to circular wait. Stop with Ctrl+C.

Part 3: Inter-Process Communication

Task 13: Unidirectional Anonymous Pipe

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pipe(fd);

    pid_t pid = fork();

    if (pid == 0) {
        close(fd[1]);
        char buffer[100];
        int n = read(fd[0], buffer, sizeof(buffer) - 1);
        if (n > 0) {
```

```
        buffer[n] = '\0';
        printf("Child read from pipe: %s\n", buffer);
    }
    close(fd[0]);
    _exit(0);
  } else {
    close(fd[0]);
    const char *msg = "Hello Child from Kernel Pipe";
    write(fd[1], msg, strlen(msg));
    close(fd[1]);
    wait(NULL);
  }
  return 0;
}
```

Task 14: POSIX Shared Memory IPC

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/wait.h>
```

```
#include <sys/stat.h>
#include <fcntl.h>

int main() {
    const char *name = "/shm_os_lab";
    int shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        perror("shm_open");
        return 1;
    }

    ftruncate(shm_fd, 1024);
    char *ptr = mmap(0, 1024, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    if (ptr == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    pid_t pid = fork();

    if (pid == 0) {
        sleep(1);
        printf("Child read from SHM: %s\n", ptr);
        _exit(0);
    } else {
```

```
        strcpy(ptr, "OS Shared Memory Payload");
        wait(NULL);
        shm_unlink(name);
    }
    return 0;
}
```

Part 4: CPU Scheduling Algorithms

The following C++ file contains implementations for Tasks 15–20.

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Process {
    int pid, arrival, burst, priority;
    int remaining, completion, turnaround, waiting;
    bool done = false;
};

void printMetrics(const vector<Process>& p) {
```

```
    cout << "PID\tArrival\tBurst\tCompletion\tTurnaround\tWait\n";
    double totalWait = 0;
    for (const auto& x : p) {
        cout << x.pid << "\t" << x.arrival << "\t" << x.burst << "\t"
             << x.completion << "\t" << x.turnaround << "\t" << x.waiting << "\n";
        totalWait += x.waiting;
    }
    cout << "Average Waiting Time: " << totalWait / p.size() << " ms\n\n";
}

// Task 15: FCFS
void schedule_fcfs(vector<Process> p) {
    sort(p.begin(), p.end(), [](const Process& a, const Process& b) {
        return a.arrival < b.arrival;
    });

    int current_time = 0;
    for (auto& proc : p) {
        if (current_time < proc.arrival)
            current_time = proc.arrival;

        proc.completion = current_time + proc.burst;
        proc.turnaround = proc.completion - proc.arrival;
        proc.waiting = proc.turnaround - proc.burst;
        current_time = proc.completion;
```

```
    }
    printMetrics(p);
}

// Task 16: SJF Non-Preemptive
void schedule_sjf(vector<Process> p) {
    int n = p.size(), completed = 0, current_time = 0;

    while (completed < n) {
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (!p[i].done && p[i].arrival <= current_time) {
                if (idx == -1 || p[i].burst < p[idx].burst)
                    idx = i;
            }
        }

        if (idx == -1) {
            current_time++;
            continue;
        }

        auto& proc = p[idx];
        proc.completion = current_time + proc.burst;
        proc.turnaround = proc.completion - proc.arrival;
```

```
        proc.waiting = proc.turnaround - proc.burst;
        proc.done = true;
        current_time = proc.completion;
        completed++;
    }
    printMetrics(p);
}

// Task 17: SRTF
void schedule_srtf(vector<Process> p) {
    for (auto& x : p) x.remaining = x.burst;

    int n = p.size(), completed = 0, current_time = 0;

    while (completed < n) {
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (p[i].arrival <= current_time && p[i].remaining > 0) {
                if (idx == -1 || p[i].remaining < p[idx].remaining)
                    idx = i;
            }
        }

        if (idx == -1) {
            current_time++;
```

```
            continue;
        }

        p[idx].remaining--;
        current_time++;

        if (p[idx].remaining == 0) {
            p[idx].completion = current_time;
            p[idx].turnaround = p[idx].completion - p[idx].arrival;
            p[idx].waiting = p[idx].turnaround - p[idx].burst;
            completed++;
        }
    }
    printMetrics(p);
}

// Task 18: Round Robin
void schedule_rr(vector<Process> p, int quantum) {
    int n = p.size(), completed = 0, current_time = 0;
    for (auto& x : p) x.remaining = x.burst;

    queue<int> q;
    vector<bool> inQueue(n, false);

    auto enqueueArrivals = [&](int time) {
```

```
    for (int i = 0; i < n; i++) {
        if (!p[i].done && !inQueue[i] && p[i].arrival <= time) {
            q.push(i);
            inQueue[i] = true;
        }
    }
};

enqueueArrivals(0);

while (completed < n) {
    if (q.empty()) {
        current_time++;
        enqueueArrivals(current_time);
        continue;
    }

    int i = q.front();
    q.pop();
    inQueue[i] = false;

    int slice = min(quantum, p[i].remaining);
    p[i].remaining -= slice;
    current_time += slice;
```

```
        enqueueArrivals(current_time);

        if (p[i].remaining > 0) {
            q.push(i);
            inQueue[i] = true;
        } else {
            p[i].done = true;
            p[i].completion = current_time;
            p[i].turnaround = p[i].completion - p[i].arrival;
            p[i].waiting = p[i].turnaround - p[i].burst;
            completed++;
        }
    }
    printMetrics(p);
}

// Task 19: Preemptive Priority Scheduling
void schedule_priority_preemptive(vector<Process> p) {
    for (auto& x : p) x.remaining = x.burst;

    int n = p.size(), completed = 0, current_time = 0;

    while (completed < n) {
        int idx = -1;
        for (int i = 0; i < n; i++) {
```

```
            if (p[i].arrival <= current_time && p[i].remaining > 0) {
                if (idx == -1 || p[i].priority < p[idx].priority)
                    idx = i;
            }
        }

        if (idx == -1) {
            current_time++;
            continue;
        }

        p[idx].remaining--;
        current_time++;

        if (p[idx].remaining == 0) {
            p[idx].completion = current_time;
            p[idx].turnaround = p[idx].completion - p[idx].arrival;
            p[idx].waiting = p[idx].turnaround - p[idx].burst;
            completed++;
        }
    }
    printMetrics(p);
}

// Task 20: MLFQ Simulation (simplified)
```

```
void schedule_mlfq(vector<Process> p) {
    int n = p.size(), completed = 0, current_time = 0;
    for (auto& x : p) x.remaining = x.burst;

    queue<int> q1, q2, q3;
    vector<bool> inQueue(n, false);

    auto enqueueArrivals = [&](int time) {
        for (int i = 0; i < n; i++) {
            if (!p[i].done && !inQueue[i] && p[i].arrival <= time) {
                q1.push(i);
                inQueue[i] = true;
            }
        }
    };

    enqueueArrivals(0);

    while (completed < n) {
        if (!q1.empty()) {
            int i = q1.front(); q1.pop(); inQueue[i] = false;
            int run = min(2, p[i].remaining);
            p[i].remaining -= run;
            current_time += run;
            enqueueArrivals(current_time);
```

```
    if (p[i].remaining > 0) {
        q2.push(i);
        inQueue[i] = true;
    } else {
        p[i].done = true;
        p[i].completion = current_time;
        p[i].turnaround = p[i].completion - p[i].arrival;
        p[i].waiting = p[i].turnaround - p[i].burst;
        completed++;
    }
} else if (!q2.empty()) {
    int i = q2.front(); q2.pop(); inQueue[i] = false;
    int run = min(4, p[i].remaining);
    p[i].remaining -= run;
    current_time += run;
    enqueueArrivals(current_time);

    if (p[i].remaining > 0) {
        q3.push(i);
        inQueue[i] = true;
    } else {
        p[i].done = true;
        p[i].completion = current_time;
        p[i].turnaround = p[i].completion - p[i].arrival;
```

```
            p[i].waiting = p[i].turnaround - p[i].burst;
            completed++;
        }
    } else if (!q3.empty()) {
        int i = q3.front(); q3.pop(); inQueue[i] = false;
        int run = p[i].remaining;
        p[i].remaining = 0;
        current_time += run;
        enqueueArrivals(current_time);

        p[i].done = true;
        p[i].completion = current_time;
        p[i].turnaround = p[i].completion - p[i].arrival;
        p[i].waiting = p[i].turnaround - p[i].burst;
        completed++;
    } else {
        current_time++;
        enqueueArrivals(current_time);
    }
  }
  printMetrics(p);
}

int main() {
  cout << "===== FCFS =====\n";
```

```
    schedule_fcfs({{1, 0, 5, 0}, {2, 1, 3, 0}, {3, 2, 8, 0}});

    cout << "===== SJF =====\n";
    schedule_sjf({{1, 0, 5, 0}, {2, 1, 8, 0}, {3, 2, 1, 0}});

    cout << "===== SRTF =====\n";
    schedule_srtf({{1, 0, 6, 0}, {2, 1, 4, 0}, {3, 3, 2, 0}});

    cout << "===== Round Robin q=2 =====\n";
    schedule_rr({{1, 0, 3, 0}, {2, 0, 2, 0}, {3, 0, 4, 0}}, 2);

    cout << "===== Preemptive Priority =====\n";
    schedule_priority_preemptive({{1, 0, 4, 2}, {2, 1, 3, 1}, {3, 2, 2, 3}});

    cout << "===== MLFQ =====\n";
    schedule_mlfq({{1, 0, 5, 0}, {2, 0, 3, 0}});

    return 0;
}
```